

Tutoriel : comprendre `\ifx` et la logique de parsing en $\text{\LaTeX}$

`\ifx` compare les *deux prochains tokens* du flux d'entrée. Contrairement à `\ifnum` ou `\ifdim`, il ne développe (« n'expand ») rien avant de comparer : il regarde la définition brute (le texte de remplacement) des séquences de contrôle, ou le code de catégorie + caractère des tokens simples. C'est cette différence qui surprend le plus souvent.

1 Exemple 1 : comparer deux séquences de contrôle identiques

```
\def\A{Bonjour}
\def\B{Bonjour}
\ifx\A\B VRAI \else FAUX \fi
```

Résultat obtenu : VRAI

Bonjour et Bonjour ont exactement le même texte de remplacement (« Bonjour »), donc `\ifx` les considère égales, même si ce sont deux macros distinctes.

2 Exemple 2 : `\ifx` ne développe pas les macros

```
\def\A{Bonjour}
\def\C{\A}          % C se developpe en "Bonjour", mais son *texte* est "\A"
\ifx\A\C VRAI \else FAUX \fi
```

Résultat obtenu : FAUX

Piège : Même si `\C` finit par se développer en « Bonjour » (comme `\A`), `\ifx` compare le *texte de remplacement immédiat* : « Bonjour » pour `\A`, mais « `\A` » pour `\C`. Ce sont des tokens différents $\Rightarrow$ FAUX.

3 Exemple 3 : comparer des tokens de caractère

```
\ifx aa VRAI \else FAUX \fi    % deux fois le caractere "a"
\ifx ab VRAI \else FAUX \fi    % "a" puis "b"
```

Résultat obtenu : « aa » → VRAI / « ab » → FAUX

Pour des caractères ordinaires (catcode 11 ou 12), `\ifx` compare le code de catégorie *et* le code caractère. « a » et « a » sont identiques ; « a » et « b » ne le sont pas.

4 Exemple 4 : tester si un argument est vide

C'est l'usage le plus fréquent en pratique (et celui qu'on a utilisé dans `\showtab`). Le principe : on capture l'argument dans une macro temporaire avec `\def`, puis on la compare à une macro qu'on sait vide.

```
\def\myempty{}
\newcommand{\testEmpty}[1]{%
  \def\tmp{#1}%
  \ifx\tmp\myempty
    Vide !%
  \else
    Non vide (contenu : #1)%
  \fi
}
\testEmpty{}
\testEmpty{Bonjour}
```

Résultat obtenu : Vide! / Non vide (contenu : Bonjour)

5 Exemple 5 : le piège des espaces

```
\testEmpty{ } % un argument contenant UN espace
```

Résultat obtenu : Non vide (contenu :)

Piège : Un argument contenant un espace n'est *pas* vide pour `\ifx` : le texte de remplacement de `\tmp` contient un token « espace », différent du texte vraiment vide de `\myempty`. Si vous voulez ignorer les espaces, il faut les supprimer avant (ex. avec `xstring`), voir Exemple 7.

6 Exemple 6 : tester si une commande est définie

Un idiome classique : comparer une séquence de contrôle à `\relax`, en utilisant `\csname` pour éviter une erreur « Undefined control sequence » si elle n'existe pas.

```
\newcommand{\testDefini}[1]{%
  \expandafter\ifx\csname #1\endcsname\relax
    #1 n'est pas definie%
  \else
```

```

    #1 est definie%
\fi
}
\testDefini{maCommandeInexistante}
\testDefini{TeX}

```

Résultat obtenu : `maCommandeInexistante` n'est pas définie / `TeX` est définie

Ici, `\expandafter` force L^AT_EX à développer `\csname...\endcsname` *avant* d'exécuter le `\ifx`, sinon celui-ci comparerait `\csname` lui-même au lieu de la commande visée. `\csname foo\endcsname` produit `\relax` si `foo` n'a jamais été définie — d'où la comparaison à `\relax`.

7 Exemple 7 : alternatives plus robustes que le `\ifx` manuel

Le test manuel « à la `\ifx` » est fragile (sensible aux espaces, aux tokens invisibles, etc.). Les packages `etoolbox` et `xstring` offrent des tests plus sûrs.

```

% etoolbox : teste si une macro est vide OU indefinie
\ifdefempty{\tmp}{Vide (etoolbox)}{Non vide (etoolbox)}

```

```

% xstring : teste si une CHAINE est vide, insensible a certains cas
\IfBlankTF{#1}{Vide (xstring)}{Non vide (xstring)}

```

Résultat obtenu : `etoolbox` → Vide

Résultat obtenu : `xstring`, argument vide → Vide / `xstring`, argument « Bonjour » → Non vide

`etoolbox` et `xstring` gèrent en interne les cas limites (espaces, macros non définies) que le `\ifx` manuel gère mal. En pratique, préférez ces outils dès que le comportement doit être fiable pour l'utilisateur final d'une commande.

8 Exemple 8 : `xparse` et les arguments optionnels absents

Pour un argument optionnel de `xparse` (type `o` ou `O{}`), il existe un test dédié, plus sûr que `\ifx`, car il distingue « argument omis » de « argument fourni mais vide » :

```

\NewDocumentCommand{\testOptionnel}{o}{%
  \IfNoValueTF{#1}
  {Argument omis}
  {Argument fourni : #1}
}

```

```
\testOptionnel
\testOptionnel[Bonjour]
```

Résultat obtenu : Argument omis / Argument fourni : Bonjour

9 Exemple 9 : application pratique — notre `\showtab`

C'est exactement la logique utilisée pour rendre la légende de `\showtab` optionnelle, avec un délimiteur `<...>` personnalisé (type `D<>{}` de `xparse`) :

```
\NewDocumentCommand{\showtab}{0{4} D<>{}} m){%
  \def\tmpcap{#2}%           on capture l'argument delimite par <...>
  \ifx\tmpcap\empty          on compare au texte vide
  \else
    \caption{#2}%           on n'appelle \caption QUE si non vide
  \fi
  % ... reste de la commande (tableau) ...
}
```

Ici, `\empty` n'a besoin d'aucune définition préalable car `LATEX` le définit déjà nativement comme une macro vide au niveau du noyau — c'est pourquoi ce test fonctionne tel quel dans le préambule d'un document standard.

10 Résumé

- `\ifx` compare des **tokens bruts**, jamais des valeurs développées.
- Deux macros sont « égales » pour `\ifx` si leur **texte de remplacement** est identique token par token — pas si elles produisent le même résultat final.
- Le test d'emptiness `\ifx\tmp\empty` est fragile aux espaces ; préférez `etoolbox` (`\ifdefempty`) ou `xstring` (`\IfBlankTF`) pour du code destiné à d'autres utilisateurs.
- Pour les arguments optionnels `xparse`, utilisez `\IfNoValueTF` plutôt que `\ifx` : c'est le mécanisme prévu pour ça, et il distingue « omis » de « vide ».
- `\expandafter` est souvent nécessaire avant `\ifx` quand on compare le résultat d'une expansion (ex. `\csname...\endcsname`).